

### EXECUTIVE SUMMARY

NutriAI is a production-grade automated diet planning application that generates a personalised 7-day meal plan in 2-3 seconds, strictly tailored to a user's clinical conditions, dietary preferences, allergen restrictions, and daily nutritional targets. The system integrates all five BAX-423 ML techniques (Sketching, Embeddings, Recommendation, Ranking, Streaming) across a 13,620-food USDA database. All four required test personas pass all six core capability checks. The app is deployed live on Streamlit Community Cloud.

### DATA PIPELINE & STATISTICS

#### Pipeline Stages

|                     |                                                                                |
|---------------------|--------------------------------------------------------------------------------|
| 1. USDA Ingest      | build_database.py - Foundation + SR Legacy + FNDDS                             |
| 2. Enrichment       | enrich_database.py - clinical tags, allergen flags, GI estimates               |
| 3. Clinical Filter  | SQL WHERE: FODMAP, GERD, GI<=55, sodium cap, 50+ name guards                   |
| 4. ANN Retrieval    | 15-dim nutrient embeddings, cosine k-NN (k=120) via sklearn                    |
| 5. Bloom Filter     | Belt-and-suspenders allergen check, 0% false-negative rate                     |
| 6. Multi-obj Rank   | 5-dimension weighted scorer (calorie, gap-fill, diversity, priority, clinical) |
| 7. Streaming Output | Python generator yields each meal live; Streamlit updates in real time         |
| 8. Gap Feedback     | Day totals feed back into gap vector; next slot adapts to what                 |

#### Key Statistics

|                      |                                      |
|----------------------|--------------------------------------|
| Foods in DB:         | 13,620                               |
| Nutrient columns:    | 21 per food                          |
| Data types:          | Foundation, SR Legacy, FNDDS         |
| Allergen filters:    | 11 types, Bloom-backed               |
| Clinical conditions: | IBS, GERD, T2DM, Hypertension        |
| Diet modes:          | 4 (vegan, vegetarian, pescat., omni) |
| BAX-423 techniques:  | 5 of 5 applicable                    |
| Generation time:     | 2-3 s end-to-end                     |
| Diversity index:     | 0.73-0.84 (Simpson's D)              |
| Personas passing:    | 4 / 4 (all 6 checks)                 |

### SIGNATURE DELIVERABLES

|                     |                                                                              |
|---------------------|------------------------------------------------------------------------------|
| "Why Excluded":     | Every filtered food returns a clinical/allergen reason string (explainer.py) |
| PDF + CSV Export:   | 10-page PDF plan   3 CSVs (meals, daily summary, exclusions) + ZIP           |
| Sub-60s Generation: | Typical cold start ~2-3s; warm (cached Bloom+embeddings) <1s                 |
| Pass/Fail Table:    | All 4 personas x 6 capabilities - see Page 3                                 |

## BAX-423 TECHNIQUES & BENCHMARKS

### T1 - SKETCHING (Bloom Filter)

*Probabilistic Data Structures for allergen safety*

11 AllergenFilterBank instances (one per allergen type). Each filter: m = 1.2M bits, k = 7 hash functions (MD5 + SHA-256 two-hash trick). Built once at session start from the full 13,620-food database. Applied AFTER SQL allergen clauses - belt-and-suspenders: SQL handles the bulk exclusion; Bloom guarantees no allergen escapes through edge-case composite foods.

|                                       |                                                        |
|---------------------------------------|--------------------------------------------------------|
| Build time (13,620 foods, 11 filters) | 873 ms (one-time per session)                          |
| Lookup time per food                  | 97.8 mcgs (O(1), k=7 hashes)                           |
| False-negative rate                   | 0.000% (by construction)                               |
| False-positive rate                   | ~0.08% (harmless: only over-excludes)                  |
| vs SQL-only safety                    | SQL alone: ~0.1% allergen escape on complex composites |

Metric

### T2 - EMBEDDINGS (Nutrient Vectors)

*Vector Representations for nutritional similarity*

15-dimensional nutrient embedding per food: [calories, protein, carbs, fat, fiber, calcium, iron, potassium, magnesium, zinc, B12, vit\_D, vit\_C, omega3\_ALA, omega3\_EPA]. Transform pipeline: weighted (B12=2.0, vit\_D=1.8, iron=1.5 to amplify scarce nutrients) -> log1p (compress concentration outliers) -> L2-normalise (enable cosine similarity). Query vector = daily nutritional gap vector (fractional remaining RDA per nutrient, sodium excluded). Index: sklearn NearestNeighbors, cosine metric, brute-force, n\_jobs=-1.

|                                |                                                  |
|--------------------------------|--------------------------------------------------|
| Index build (5,000 foods)      | ~60 ms                                           |
| Query time (k=120)             | <5 ms                                            |
| Gap-fill score: ANN top-5      | 5.26 (nutrient relevance index)                  |
| Gap-fill score: random 5 foods | 1.60 (baseline)                                  |
| ANN improvement over random    | +230% - retrieves foods 3.3x more aligned to gap |

Metric

### T3 - RECOMMENDATION (Content-Based ANN)

*Recommendation Systems - adaptive gap-filling content filter*

Content-based filtering: the query IS the user's current nutritional state. After each meal is assigned, day\_totals updates, compute\_gap\_vector() recalculates the fractional remaining RDA, and the next ANN query reflects what was actually consumed - creating a closed adaptive feedback loop across all 21 meal slots. k=120 candidates retrieved, then Bloom-filtered, slot-suitability-filtered, ranked.

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| Adaptive iterations per plan   | 21 (gap vector recalculated after every meal) |
| Candidates retrieved per slot  | 120 (cosine ANN from candidate pool)          |
| Pool sizes (typical)           | 1,782 (Mei) to 3,805 (Ravi)                   |
| Coverage vs fixed-gap baseline | Adaptive: 4/7 RDA days (Priya) vs 0/7 fixed   |

Metric

### T4 - RANKING (Multi-Objective Weighted Score)

*Ranking & Learning-to-Rank - five-dimension weighted scorer*

5 scoring dimensions combined as weighted sum -> final score [0,1]. W1=0.20 Calorie Fit: Gaussian on delivered\_kcal/slot\_target, sigma=0.5, with per-day budget guard (<=110% of daily target) and per-slot calorie density cap. W2=0.32 Nutrient Gap-Fill: fractional RDA fill weighted by relative gap urgency. W3=0.23 Diversity: Simpson's D penalty 1/(1+count\*decay) per food group; legumes decay=0.30, vegetables=0.45 to prioritise nutritionally important groups. W4=0.17 Priority Micro: nutrient-calibrated thresholds per priority nutrient. W5=0.08 Clinical Bonus: condition-specific food group and nutrient rewards.

|                                       |                                                     |
|---------------------------------------|-----------------------------------------------------|
| Diversity - multi-objective ranker    | 0.816 (Simpson's D, Priya; range 0.78-0.87)         |
| Diversity - random selection baseline | 0.617 (same food pool, no ranking)                  |
| Diversity improvement                 | +0.199 (+32% better food-group spread)              |
| Calorie adherence (all personas)      | 0/7 days exceed 110% of target (after budget guard) |
| RDA coverage: Priya "days met"        | 4/7 days (vs 0/7 before gap-fill reweighting)       |

Metric

### T5 - STREAMING (Progressive Plan Generation)

*Streaming & Real-Time Processing - live meal-by-meal delivery*

generate\_plan\_stream() is a Python generator. Each of 21 meal slots yields a typed event dict {type:"meal", day, slot, meal, elapsed\_s} immediately after selection. Streamlit iterates the generator and updates the progress bar + live description for each event. The user sees meals appearing one-by-one rather than waiting for the entire plan to complete.

|                                   |                                                       |
|-----------------------------------|-------------------------------------------------------|
| Time to first meal (cold start)   | ~2.1 s (includes Bloom build + embedding index)       |
| Total plan time (cold start)      | ~3.0 s                                                |
| Time to first meal (warm, cached) | <200 ms (Streamlit @st.cache_resource)                |
| Events yielded per plan           | 22 (21 meals + 1 "complete" event)                    |
| vs batch (wait for all 21)        | User sees plan building live; 17% faster first result |

Metric

PERSONA PASS / FAIL TABLE - All 4 Test Personas x 6 Core Capabilities

| Priya (28y F, 1800 kcal) |                            | Ravi (38y M, 2200 kcal) |                            |
|--------------------------|----------------------------|-------------------------|----------------------------|
| Diet:                    | Vegetarian                 | Diet:                   | Non-veg / No pork          |
| Condition:               | IBS                        | Condition:              | GERD                       |
| Allergen:                | Dairy                      | Allergen:               | Gluten (Celiac)            |
| Priority Micros:         | Iron, Calcium, Vit D       | Priority Micros:        | B12, Zinc, Mg              |
| Weight / Protein floor:  | 58 kg => >41 g/day protein | Weight / Protein floor: | 80 kg => >56 g/day protein |

| Mei (45y F, 1600 kcal)  |                            | James (52y M, 2000 kcal) |                            |
|-------------------------|----------------------------|--------------------------|----------------------------|
| Diet:                   | Vegan                      | Diet:                    | Pescatarian                |
| Condition:              | Type 2 Diabetes            | Condition:               | Hypertension               |
| Allergen:               | Tree Nuts                  | Allergen:                | Soy                        |
| Priority Micros:        | B12, Iron, Zinc            | Priority Micros:         | Potassium, Mg, Omega-3     |
| Weight / Protein floor: | 62 kg => >43 g/day protein | Weight / Protein floor:  | 88 kg => >62 g/day protein |

| Capability                                    | Priya | Ravi  | Mei   | James |
|-----------------------------------------------|-------|-------|-------|-------|
| C1 Clinical Condition Filtering               | PASS  | PASS  | PASS  | PASS  |
| FODMAP, low-acid, GI<=55, DASH sodium cap     |       |       |       |       |
| C2 Allergy Detection & Exclusion              | PASS  | PASS  | PASS  | PASS  |
| SQL + Bloom filter; zero false negatives      |       |       |       |       |
| C3 Dietary Preference Handling                | PASS  | PASS  | PASS  | PASS  |
| Vegetarian / vegan / pescatarian / non-veg    |       |       |       |       |
| C4 Diversity Engine                           | 0.834 | 0.816 | 0.730 | 0.839 |
| No food repeated; Simpson's D >= 0.70         |       |       |       |       |
| C5 Macro & Micro Analysis                     | PASS  | PASS  | PASS  | PASS  |
| 21 nutrients tracked; RDA gaps flagged at 80% |       |       |       |       |
| C6 Sub-60s Generation                         | 3.0s  | 2.9s  | 2.1s  | 2.1s  |
| Cold ~2s; warm (cached) <1s                   |       |       |       |       |

KEY OUTCOME METRICS (per persona)

| Metric                  | Priya    | Ravi    | Mei     | James    |
|-------------------------|----------|---------|---------|----------|
| Generation time         | 3.0 s    | 2.9 s   | 2.1 s   | 2.1 s    |
| Candidate pool size     | 1,960    | 3,805   | 1,782   | 2,480    |
| Diversity (Simpson's D) | 0.834    | 0.816   | 0.730   | 0.839    |
| Days >= 80% RDA (+)     | 4 / 7    | 0 / 7 * | 1 / 7 * | 0 / 7 *  |
| Calorie adherence       | Avg 95%  | Avg 97% | Avg 91% | Avg 94%  |
| Days >110% calories     | 0 / 7    | 0 / 7   | 0 / 7   | 0 / 7    |
| Iron >= 80% RDA daily   | PASS 6/7 | -       | -       | -        |
| Fiber >= 25g/day        | -        | -       | 3-5 / 7 | -        |
| Potassium >= 80% RDA    | -        | -       | -       | 4 / 7    |
| Sodium <= 1500mg/day    | -        | -       | -       | 0 / 7 ** |

(+) "Days >= 80% RDA" excludes Vitamin D (primarily sunlight-dependent; food alone <5% of 600 IU RDA), Omega-3 ALA (plant sources excluded as non-standalone-meal items), and Sodium (a cap, not a floor). \* Structural constraints: Ravi fiber 38g/day near-impossible with GERD+gluten-free; Mei calories/fat on vegan+T2DM+1600 kcal; James fat/fiber on pescatarian+hypertension+soy-free. \*\* James sodium: cumulative multi-slot overshoot; individual foods all pass is\_high\_sodium=0 filter.

## ADAPTIVE LEARNING - CLOSED-LOOP GAP FEEDBACK

NutriAI implements within-plan adaptive learning: after each meal is assigned, `compute_day_totals()` aggregates actual nutrients delivered, and `compute_gap_vector()` recalculates fractional remaining RDA needs (Eq:  $\text{gap}[n] = \max(0, (\text{RDA} - \text{actual}) / \text{RDA})$ ). The ANN query and the ranker's gap-fill score both receive this updated vector before selecting the next slot. This creates a 21-iteration closed feedback loop: each meal selection observes what was already consumed and adapts its priorities accordingly.

Impact: Priya's "days meeting 80% RDA" improved from 0/7 (fixed gap) to 4/7 (adaptive) after correcting the gap normalisation. James potassium improved from 1/7 to 4/7. Mei fiber improved from 1/7 to 3-5/7. The adaptive loop accounts for the largest single improvement in RDA coverage across all personas.

## DEPLOYMENT & SUBMISSION

|                      |                                                                                                                                                                        |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Live App URL:</b> | <a href="https://nutriai-bax423-msba.streamlit.app">https://nutriai-bax423-msba.streamlit.app</a> (Streamlit Community Cloud)                                          |
| <b>Tech Stack:</b>   | Python 3.13, Streamlit, SQLite, scikit-learn, fpdf2, plotly, bitarray                                                                                                  |
| <b>Database:</b>     | 13,620 foods   <code>nutriai_foods.db</code>   ~10 MB SQLite   in repo                                                                                                 |
| <b>Setup:</b>        | <code>pip install -r requirements.txt -&gt; streamlit run app.py</code>                                                                                                |
| <b>Rebuild DB:</b>   | <code>python build_database.py</code> (13 min, USDA API) + <code>python enrich_database.py</code> (24s)                                                                |
| <b>Tests:</b>        | <code>python test_personas.py</code> - all 4 personas, 6 capability checks, ~5 s                                                                                       |
| <b>ZIP Contents:</b> | <code>code/</code> (full src), <code>data/</code> ( <code>nutriai_foods.db</code> snapshot), <code>brief.pdf</code> , <code>prompts.md</code> , <code>README.md</code> |

## KNOWN LIMITATIONS (Honest Assessment)

### Ravi 0/7 RDA days:

Fiber RDA of 38 g/day (male) is clinically near-impossible with GERD + gluten-free combined. The pipeline correctly avoids bran/psyllium (excluded as supplements) and acidic high-fiber foods (GERD triggers). This reflects real dietary constraint, not a pipeline failure.

### James cumulative sodium:

The ranker excludes individually high-sodium foods (>400 mg/100 g) via `is_high_sodium=0` SQL flag, but three moderate-sodium foods combined can exceed the 1,500 mg/day DASH cap. Fix: pass rolling day sodium into the gap vector as a soft penalty once budget > 60%.

### Vitamin D & Omega-3 ALA:

Food-only Vitamin D achieves <5% of the 600 IU RDA for dairy-free profiles. Omega-3 ALA requires nuts/seeds/oils which are excluded as non-standalone meal items. Both are excluded from the "days met" UI metric with a footnote; supplementation is recommended.

### Calorie variation +/-10%:

Daily calorie delivery varies +/-5-10% around the user target. A per-slot calorie density cap (scaled to user calorie target) and a per-day budget guard (<=110%) prevent large outliers. Average adherence is 91-97% across all personas. Exact daily calorie matching would require further narrowing the calorie scoring sigma or scaling serving sizes proportionally for sub-2000 kcal profiles.

### No user feedback persistence:

Adaptive learning operates within a single plan generation. User preferences (liked/disliked foods) are not persisted across sessions. A future version could store ratings in a local JSON and use them to bias food group weights in subsequent generations.

## TECHNICAL REFLECTION

The two most impactful design decisions were (1) the fractional gap-vector normalisation and (2) the calorie-proportional serving size. Before normalisation, absolute units caused calcium (1,000 mg RDA) to dominate fiber (25 g) by 40x in the gap-fill score, making the ANN query and ranker effectively optimise only for calcium. After fractional normalisation ( $\text{gap} = \max(0, (\text{RDA} - \text{actual}) / \text{RDA})$ ), all 15 nutrients compete equally. This alone moved Priya from 0/7 to 4/7 days meeting RDA.

The five BAX-423 techniques form a sequential pipeline rather than isolated components: embeddings generate the query space, ANN recommendation retrieves candidates in that space, Bloom filter provides a safety guarantee, ranking selects the best among candidates, and streaming delivers results progressively. The adaptive gap-feedback loop ties them together into a system that improves its own recommendations as the day's plan is built.